

Digitize Metro Ticket Booking

ServiceNow Service Catalog Implementation

Project Documentation

Prepared for: ServiceNow PDI (Australia Instance)

Date: August 2026

1. Introduction

Project Title

Digitize Metro Ticket Booking using ServiceNow

Submitted By

Naidu Archana

Role: ServiceNow Developer (Individual Project) — responsible for end-to-end design, catalog build, scripting, testing, and documentation.

2. Project Overview

Purpose

This project implements a metro e-ticketing system as a ServiceNow Service Catalog item. It allows passengers to either book a QR-based metro ticket for a single or return journey, or recharge an existing metro smart card — replicating the real-world dual-service counter experience (buy a ticket vs. top up a travel card) inside the ServiceNow Service Portal.

Features

- Single catalog item offering two branching services: Book a Metro Ticket and Recharge Smart Card
- Dynamic form fields shown/hidden based on the selected service using UI Policies
- Journey details capture: Starting Station, Destination, Number of Passengers, Journey Type (Single/Return)
- Automatic fare calculation based on number of passengers and journey type via a Client Script
- Multiple payment modes (Card, Cash, UPI, Others) with a conditional free-text field for 'Others'
- Smart card recharge capture: Card Number, Card Name, Recharge Amount
- QR code generation for the recharged smart card, displayed to the user on submission
- Automatic Requested Item (RITM) and Request (REQ) generation with full request tracking
- Additional Details view on the Requested Item form showing all submitted values

3. Architecture

Platform

Built entirely on the ServiceNow platform using native Service Catalog and Service Portal capabilities — no external frontend/backend stack is used.

Service Portal (Frontend)

- Service Portal (/sp) used as the end-user facing interface
- Catalog item rendered via the standard Service Portal 'Catalog Item' widget
- Catalog Client Scripts (onChange, onSubmit) used for dynamic behavior in the browser

- UI Policies used to show/hide and mandate fields based on user selection

Platform Layer (Backend)

- Service Catalog: sc_cat_item — 'Book A Metro Ticket'
- Variables: sc_item_option / item_option_new — capture journey, passenger, and recharge details
- Requested Item (sc_req_item) and Request (sc_request) — auto-generated on order submission
- Question Choices — used to define the two branching service options (Value: recharge_card / book_ticket)

Database

Uses ServiceNow's native platform tables (sc_cat_item, sc_req_item, sc_request, sc_item_option_mst, sc_item_option_mst_value) — no external database is required.

Third-Party Integration

- QR Server API (api.qrserver.com) — used to generate the QR code image for the recharged smart card, called directly from the client script

4. Setup Instructions

Prerequisites

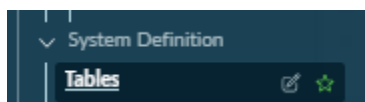
- A ServiceNow Personal Developer Instance (PDI) — Australia data residency instance used for this project
- Admin (or catalog_admin) role access
- Access to Service Catalog > Catalog Definitions module

Installation / Configuration Steps

1. Log in to the ServiceNow PDI as System Administrator.
2. Copy the instance domain, e.g. https://dev190678.service-now.com
3. Append /sp to the URL to access the Service Portal, e.g. https://dev190678.service-now.com/sp
4. Navigate to Service Catalog > Catalog Definitions > Maintain Items > New to create the catalog item 'Book A Metro Ticket'.
5. Add variables, UI Policies, and Catalog Client Scripts as detailed in Section 7 (API / Scripts) below.

5. Step by step guide

Create Tables



Tables>new>

Table Metro Station's Details

* Label Metro Station's Details

* Name u_metro_station_s_details

Application Global

Remote Table

Delete

Update

Delete All Records

Columns

Controls

Application Access

Table Columns for text

1 to 11 of 11

New

Dictionary Entries

Column label	Type	Reference	Max length	Default value	Display
Updates	Integer	(empty)	40	false	
Distance Km	Decimal	(empty)	20	false	
Station Name	String	(empty)	40	true	
Route Line	Choice	(empty)	40	false	
Updated	Date/Time	(empty)	40	false	
Sys ID	Sys ID (GUID)	(empty)	32	false	
Created by	String	(empty)	40	false	
Station Code	String	(empty)	20	false	
Updated by	String	(empty)	40	false	
Created	Date/Time	(empty)	40	false	
Fare Zone	Integer	(empty)	40	false	
Insert a new row...					

Delete

Update

Delete All Records

Related Links

[Form Builder](#)
[Design Form](#)
[Layout Form](#)
[Layout List](#)
[Show Form](#)
[Show List](#)
[Show Schema Map](#)
[Add to Service Catalog](#)
[Run Point Scan](#)
[Export REST API](#)
[ISN UriList Collection Dictionary Entry](#)
[ISN UriList Versions List](#)

Access Controls (4)

Security Data Filters

Labels (1)

Database Indexes (1)

Table Subscription Configuration (1)

Updated

1 to 12 of 12

Actions on selected rows...

Access Controls

Name	Decision Type	Operation	Type	Active	Updated by	Updated
u_metro_station_s_details	Allow If	delete	record	true	admin	2026-07-27 13:46:53
u_metro_station_s_details	Allow If	read	record	true	admin	2026-07-27 13:46:53
u_metro_station_s_details	Allow If	write	record	true	admin	2026-07-27 13:46:53
u_metro_station_s_details	Allow If	create	record	true	admin	2026-07-27 13:46:53

Submit
create another table

Table Metro Database

* Label Metro Database

* Name u_metro_database

Application Global

Remote Table

Delete

Update

Delete All Records

Columns

Controls

Application Access

Table Columns for text

1 to 12 of 12

New

Dictionary Entries

Column label	Type	Reference	Max length	Default value	Display
Created	Date/Time	(empty)	40	false	
Mode of Payment	String	(empty)	40	false	
Updates	Integer	(empty)	40	false	
Updated by	String	(empty)	40	false	
User Details	Reference	User	32	false	
Recharge Amount	String	(empty)	40	false	
Updated	Date/Time	(empty)	40	false	
Created by	String	(empty)	40	false	
Created	Date/Time	(empty)	40	false	
Sys ID	Sys ID (GUID)	(empty)	32	false	
Smart Card Name	String	(empty)	40	false	
Smart Card Number	String	(empty)	20	false	
Insert a new row...					

Delete

Update

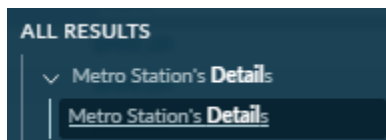
Delete All Records

Related Links

[Form Builder](#)
[Design Form](#)
[Layout Form](#)
[Layout List](#)
[Show Form](#)
[Show List](#)
[Show Schema Map](#)
[Add to Service Catalog](#)
[Run Point Scan](#)
[Export REST API](#)
[ISN UriList Collection Dictionary Entry](#)
[ISN UriList Versions List](#)

Access Controls (4) Security Data Filters Labels (1) Database Indexes (2) Table Subscription Configuration (1)							
Updated Search							
Access Controls							
☐	Name	Decision Type	Operation	Type	Active	Updated by	Updated
	u_metro_database	Allow If	read	record	true	admin	2026-07-27 20:18:06
	u_metro_database	Allow If	write	record	true	admin	2026-07-27 20:18:06
	u_metro_database	Allow If	delete	record	true	admin	2026-07-27 20:18:06
	u_metro_database	Allow If	create	record	true	admin	2026-07-27 20:18:05

Go to all navigation>



Create records>

Metro Station's Details Fare Zone Search					
All					
☐	Station Name	Station Code	Distance Km	Fare Zone	Route Line
	Search	Search	Search	Search	Search
	Raidurg	RDG		8	
	Nagole	NGL		9	
	Miyapur	MYP		12	
	Ameerpet	AMP		5.2	1
	LB Nagar	LBN		10	2

Catalog Structure

Since this is a ServiceNow implementation rather than a traditional client/server codebase, the equivalent of a 'folder structure' is the catalog configuration structure:

Catalog Item

- Book A Metro Ticket

1.create a catalog item

Catalog Client Scripts (3 total)

- FareCalculation — onChange — auto-calculates fare based on passengers/journey type

The screenshot shows the configuration for the 'FareCalculation' script. The 'Name' is 'FareCalculation', 'Applies to' is 'A Catalog Item', 'Active' is checked, and 'UI Type' is 'All'. The 'Application' is 'Global', 'Type' is 'onChange', 'Catalog item' is 'Book A Metro Ticket', and 'Variable name' is 'type_of_journey'. The script is active on a Catalog Item view. The script code is as follows:

```
1 function onChange(control, oldValue, newValue, isLoading) {
2
3     if (isLoading || newValue == '') {
4         return;
5     }
6
7     var source = g_form.getValue('starting_from');
8     var destination = g_form.getValue('going_to');
9     var numPassengers = parseInt(g_form.getValue('no_of_passengers'));
10
11     // Ameerpet to L B Nagar
12     if (newValue == 1) {
13         if (source == "9e97bf46c396471017eb160ed4013197" && destination ==
14             '0d296c96c31e471017eb160ed40131f0') {
15             g_form.setValue('amount_for_single_journey', numPassengers * 50);
16             g_form.clearValue('amount_including_return');
17         }
18     } else if (newValue == 2) {
19         if (source == "9e97bf46c396471017eb160ed4013197" && destination ==
20             '0d296c96c31e471017eb160ed40131f0') {
21             g_form.clearValue('amount_for_single_journey');
22             g_form.setValue('amount_including_return', 2 * (numPassengers * 50));
23         }
24     }
25 }
```

Below the script editor, there are 'Update' and 'Delete' buttons, and a 'Related Links' section with a link to 'Run Point Scan'. A 'Versions' table is also present, showing the history of updates.

Name	Recorded at	State	Source	Reverted from
catalog_script_client_122e3706c3d6471017eb160ed4013150	2026-07-27 18:18:30	Current	Update Set: Default	(empty)
catalog_script_client_122e3706c3d6471017eb160ed4013150	2026-07-27 18:08:04	Previous	Update Set: Default	(empty)
catalog_script_client_122e3706c3d6471017eb160ed4013150	2026-07-27 14:45:59	Previous	Update Set: Default	(empty)

Script:

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }
    var source = g_form.getValue('starting_from');
    var destination = g_form.getValue('going_to');
    var numPassengers = parseInt(g_form.getValue('no_of_passengers'));
    //Ameerpet to L B nagar and taken sys_ids of both the stations
    if (newValue == 1) {
        if (source == "7d8a5fcb83da0f507fa99c50ceaad363" && destination
        == '219c970b831e0f507fa99c50ceaad328') {
            g_form.setValue('amount_for_single_journey', numPassengers *
50);
            g_form.clearValue('amount_including_return');
        }
    }
}
```

```

    } else if (newValue == 2) {

        if (source == "7d8a5fcb83da0f507fa99c50ceaad363" && destination
== '219c970b831e0f507fa99c50ceaad328') {
            g_form.clearValue('amount_for_single_journey');
            g_form.setValue('amount_including_return', 2 (numPassengers *
50));
        }
    } // ameerpet to Miyapur
    if (newValue == 1) {
        if (source == "7d8a5fcb83da0f507fa99c50ceaad363" && destination
== '307cdbc7831e0f507fa99c50ceaad3f7') {
            g_form.setValue('amount_for_single_journey', numPassengers *
60);

            g_form.clearValue('amount_including_return');
        }
    } else if (newValue == 2) {
        if (source == "7d8a5fcb83da0f507fa99c50ceaad363" && destination
== '307cdbc7831e0f507fa99c50ceaad3f7') {
            g_form.clearValue('amount_for_single_journey');
            g_form.setValue('amount_including_return', 2 (numPassengers *
60));
        }
    } // ameerpet to Nagole
    if (newValue == 1) {
        if (source == "7d8a5fcb83da0f507fa99c50ceaad363" && destination
== 'deebdb07831e0f507fa99c50ceaad3e0') {
            g_form.setValue('amount_for_single_journey', numPassengers *
55);

            g_form.clearValue('amount_including_return');
        }
    } else if (newValue == 2) {
        if (source == "7d8a5fcb83da0f507fa99c50ceaad363" && destination
== 'deebdb07831e0f507fa99c50ceaad3e0') {
            g_form.clearValue('amount_for_single_journey');
            g_form.setValue('amount_including_return', 2 (numPassengers *
55));
        }
    } // ameerpet to Raiadurg
    if (newValue == 1) {
        if (source == "7d8a5fcb83da0f507fa99c50ceaad363" && destination
== 'd4ac174b831e0f507fa99c50ceaad3c4') {

```



```

40);

        g_form.setValue('amount_for_single_journey', numPassengers *

        g_form.clearValue('amount_including_return');

    }

    } else if (newValue == 2) {

        if (source == "7d8a5fcb83da0f507fa99c50ceaad363" && destination
== 'd4ac174b831e0f507fa99c50ceaad3c4') {

            g_form.clearValue('amount_for_single_journey');

            g_form.setValue('amount_including_return', 2 (numPassengers *

40));

        }

    }

}

```

- FieldValidation — onChange — validates form input

FieldValidation configuration interface showing various settings and a script editor.

Script (script) editor content:

```

1 function onChange(control, oldValue, newValue, isLoading) {
2   if (isLoading || newValue == '') {
3     return;
4   }
5   if (isNaN(newValue)) {
6     alert('please enter the values in number format only');
7   }
8 }

```

Related Links: Run Point Scan

Update Versions table:

Name	Recorded at	State	Source	Reverted from
catalog_script_client_2c4f7132c3968b1017eb160e4013151	2026-07-29 13:23:43	Current	Update Set: Default	(empty)

Script:

```

function onChange(control, oldValue, newValue, isLoading) {
  if (isLoading || newValue == '') {
    return;
  }
  if (isNaN(newValue)) {

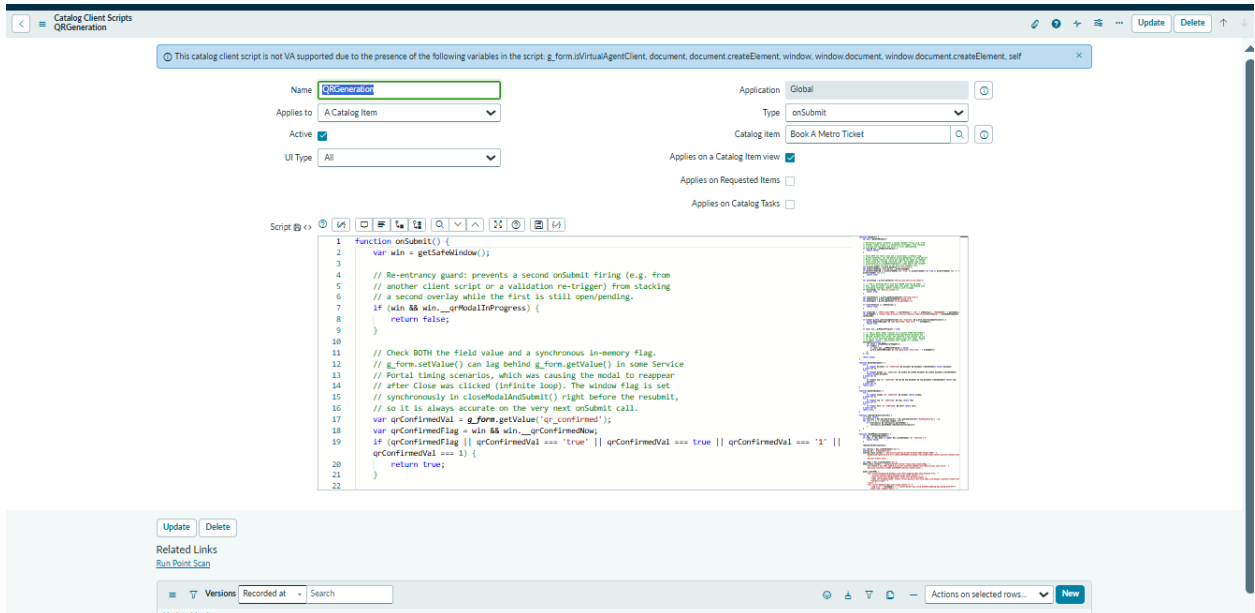
```

```

    alert('please enter the values in number format only');
  }
}

```

- QR Generation — onSubmit — generates and displays the QR code for smart card recharge



```

function onSubmit() {
  var work = g_form.getValue('what_do_you_want_to_do_today');
  if (work == 'book_qr_ticket' && g_form.getValue('qr_confirmed') !=
'true') {
    var sysId = g_form.getUniqueValue();
    var dummyURL = 'https://example.com/ticket?id=' + sysId;
    spModal.open({
      title: "Scan Your QR Here",
      widget: "qr_ticket_widget",
      widgetInput: {
        ticketUrl: dummyURL
      }
    }).then(function() {
      g_form.setValue('qr_confirmed', 'true');
      g_form.submit();
    });
    return false;
  }
  return true;
}

```

UI Policies

- Show ticket-booking fields when 'Book a Metro Ticket' is selected
- Show recharge fields when 'Recharge Smart Card' is selected
- Show 'Amount Including Return' when Journey Type = Return
- Show 'Enter payment mode' when Mode of Payment = Others

Create catalog UI policies

[\[SN Utilis\] Versions \(126\)](#)

Variables (14) Variable Sets **Catalog UI Policies (3)** Catalog Client Scripts (4) Available For Not Available For Categories (1) Catalogs (1) Catalog Data Lookup Definitions Related Articles Related Catalog Items Assigned Topics

Order - Search

Actions on selected rows... **New**

Catalog item = Book A Metro Ticket

☐	Short description	Variable set	Conditions	Reverse if false	On load	Inherit	Updated	Order
☐	Show payment mode field when Others selected	(empty)		true	true	false	2026-07-29 13:08:00	100
☐	Reset QR confirmation on revisit	(empty)		false	false	false	2026-07-29 22:09:47	100
☐	Show ticket fields when Book QR Ticket selected	(empty)		true	true	false	2026-07-29 22:30:48	100

<< 1 to 3 of 3 >>

Catalog UI Policy
Show payment mode field when Others selected

Applies to: A Catalog Item
Catalog item: Book A Metro Ticket
Application: Global
Active: ☒

* Short description: Show payment mode field when Others selected

When to Apply | Script

Catalog Conditions: **Add Filter Condition** **Add OR Clause**
mode_of_payment is Others AND OR

Applies on a Catalog Item view ☒
Applies on Catalog Tasks ☐
Applies on Requested Items ☐

On load ☒
Reverse if false ☒

Update **Delete**

Related Links
[Run Point Scan](#)
[\[SN Utilis\] Versions \(1\)](#)

Catalog UI Policy Actions Order - Search

UI policy = Show payment mode field when Others selected

☐	Name	Read only	Mandatory	Visible	Order
☐	enter_payment_mode	Leave alone	True	True	100

<< 1 to 1 of 1 >>

Catalog UI Policy
Reset QR confirmation on revisit

Applies to: A Catalog Item
Catalog item: Book A Metro Ticket
Application: Global
Active: ☒

* Short description: Reset QR confirmation on revisit

When to Apply | Script

Catalog Conditions: **Add Filter Condition** **Add OR Clause**
what_do_you_want_to_do_today is anything AND OR

Applies on a Catalog Item view ☒
Applies on Catalog Tasks ☐
Applies on Requested Items ☐

On load ☐
Reverse if false ☐

Update **Delete**

Related Links
[Run Point Scan](#)
[\[SN Utilis\] Versions \(1\)](#)

Catalog UI Policy Actions Order - Search

UI policy = Reset QR confirmation on revisit

☐	Name	Read only	Mandatory	Visible	Order
No records to display					

Catalog UI Policy
 Show ticket fields when Book QR Ticket selected

Applies to: A Catalog Item
 Application: Global
 Catalog Item: Book A Metro Ticket
 Active: ☒

Short description: Show ticket fields when Book QR Ticket selected

When to Apply | Script

Catalog Conditions: Add Filter Condition Add OR Clause
 what_do_you_want_to_do_today is Book QR Ticket AND OR

Applies on a Catalog Item view: ☒
 Applies on Catalog Tasks: ☐
 Applies on Requested Items: ☐

On load: ☒
 Reverse if false: ☒

Update Delete

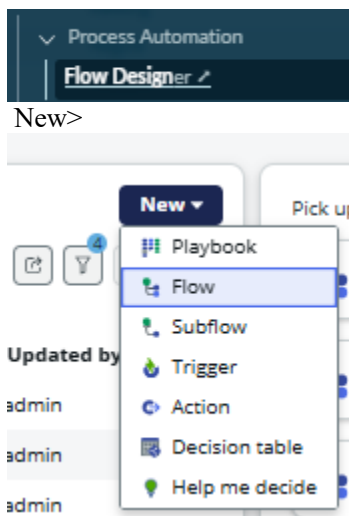
Related Links
[Run Point Scan](#)
[SN Utilis Versions \(2\)](#)

Catalog UI Policy Actions Order Search

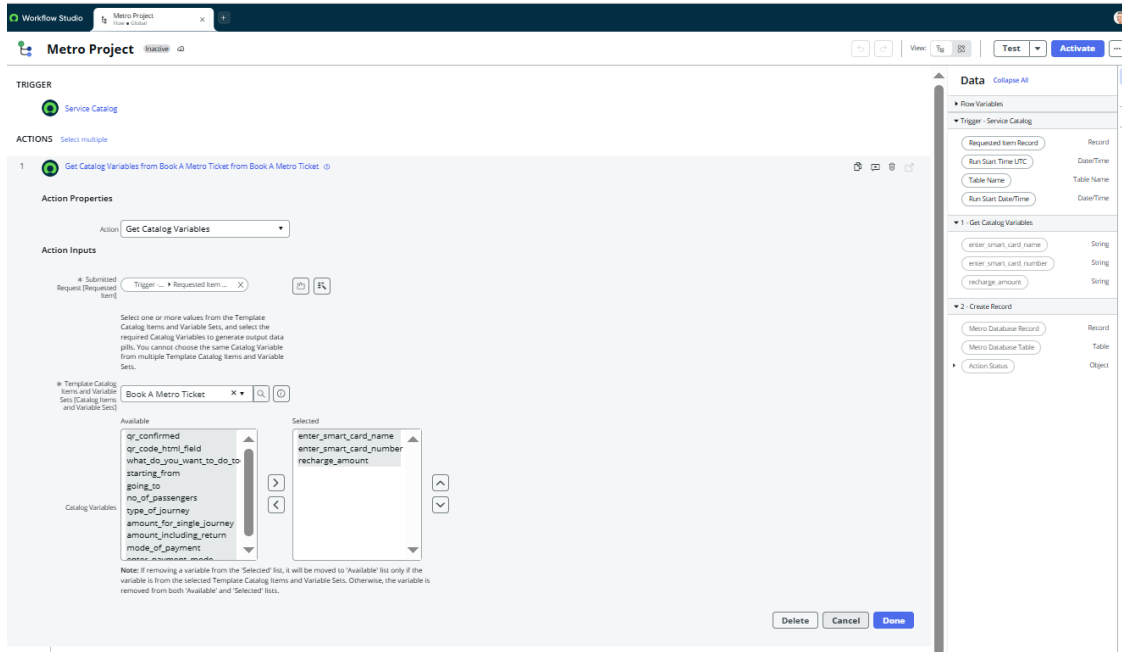
Name	Read only	Mandatory	Visible	Order
amount_for_single_journey	Leave alone	Leave alone	True	100
recharge_amount	Leave alone	False	False	100
going_to	Leave alone	True	True	100
amount_including_return	Leave alone	Leave alone	True	100
type_of_journey	Leave alone	True	True	100
enter_smart_card_name	Leave alone	False	False	100
no_of_passengers	Leave alone	True	True	100
enter_smart_card_number	Leave alone	False	False	100
mode_of_payment	Leave alone	True	True	100
starting_from	Leave alone	True	True	100

1 to 10 of 10

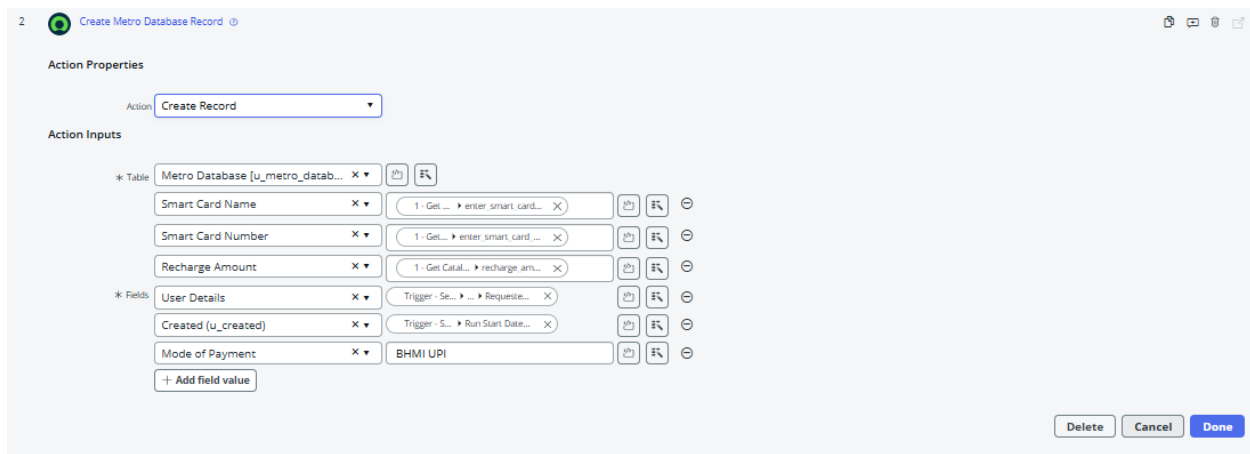
Create flow



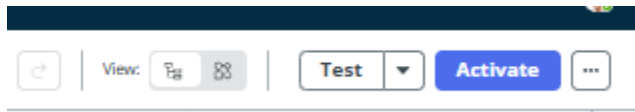
Create trigger



Click done
Create action



Click done and activate

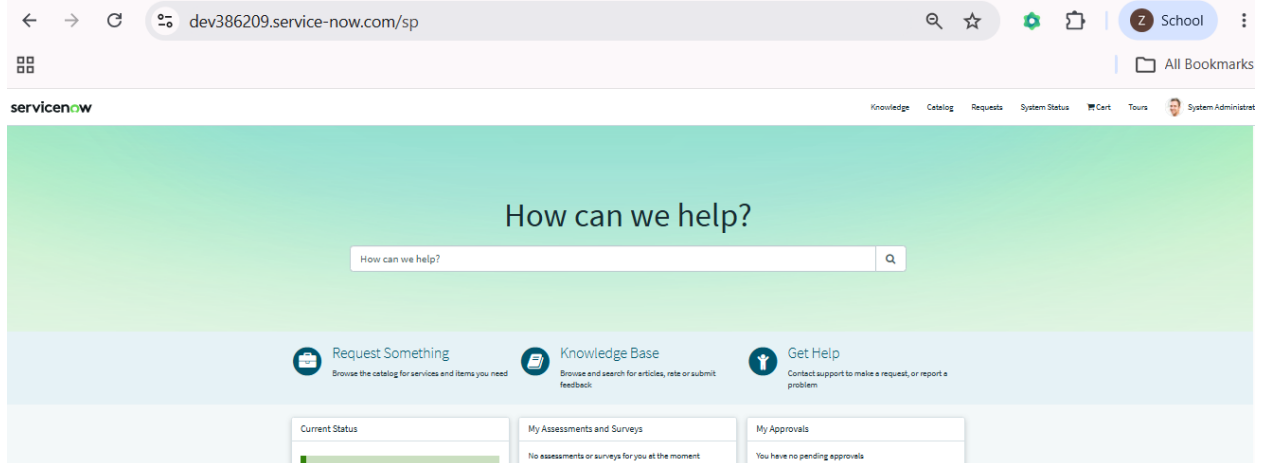


6. Running the Application

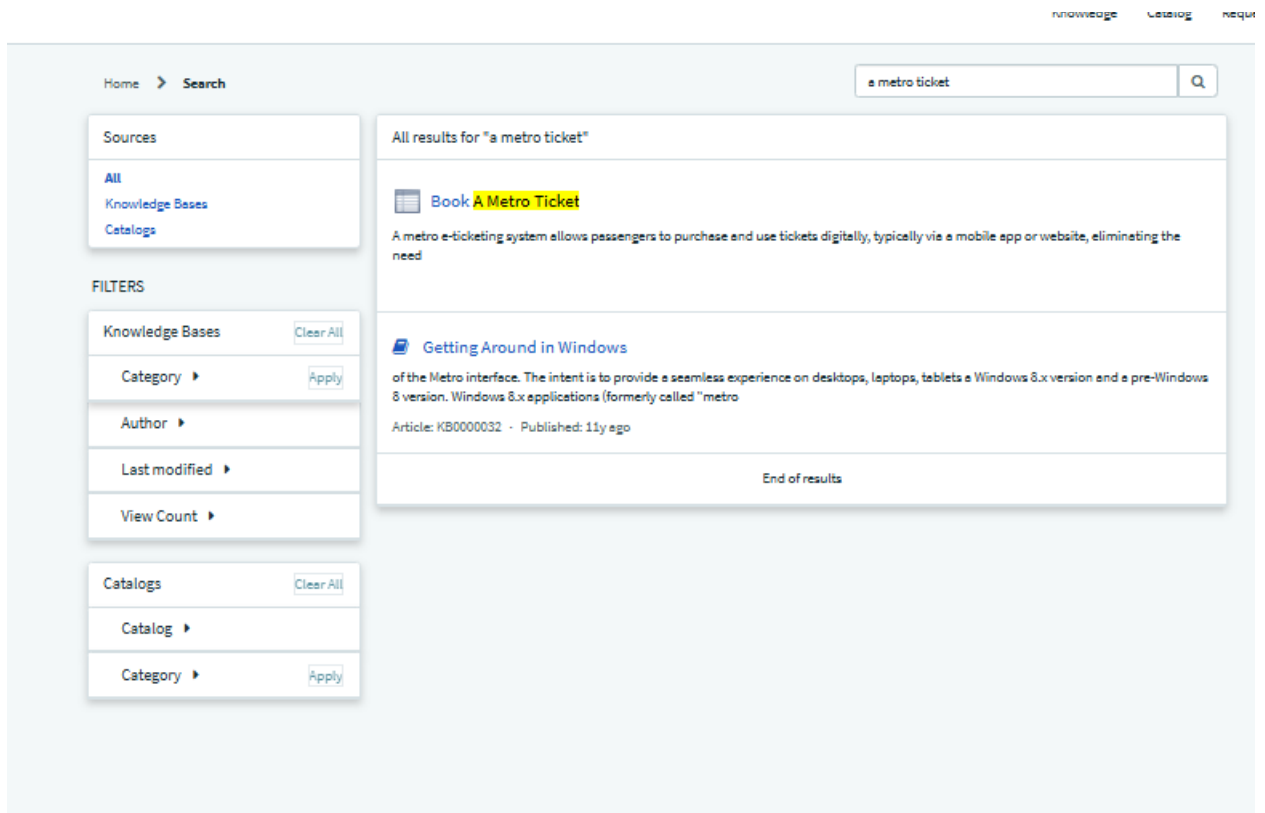
As a ServiceNow catalog item, there is no separate frontend/backend server to start. The application runs directly within the ServiceNow instance:

1. Open the Service Portal:

<https://<your-instance>.service-now.com/sp>



2. Search for "Book A Metro Ticket" in the catalog search bar.



3. Fill in the required details and click Order Now to submit.

Home > All Catalogs > Service Catalog > Office > Services > Book A Metro Ticket
Search Catalog

Book A Metro Ticket

A metro e-ticketing system allows passengers to purchase and use tickets digitally, typically via a mobile app or website, eliminating the need

*** Indicates required**

***What do you want to do Today?**

☐ --None--
☒ Recharge Metro Card
☐ Book QR Ticket

***Enter Smart Card Number**

e.g. 1234567890

***Enter Smart Card Name**

***Recharge Amount**

Quantity: 1

Delivery Time: 2 Days

Add to Cart
 Save as Draft

Order Now

Required information

Enter Smart Card Number

Enter Smart Card Name

Recharge Amount

7. API Documentation

External API Used

QR Server API — used to generate a scannable QR code image representing the recharged smart card.

```
GET https://api.qrserver.com/v1/create-qr-code/
?size=250x250
&data=<encoded card number, amount, and reference ID>
```

Called from the 'QR Generation' Catalog Client Script (onSubmit) when the user selects Recharge Smart Card.

Internal Platform APIs

Function	Description
g_form.getValue()	Reads the current value of a catalog variable
g_form.setValue()	Sets a catalog variable value (used for auto fare calculation)
g_form.getUniqueValue()	Retrieves the RITM sys_id used as a QR reference number

8. Authentication

Authentication and authorization are handled natively by the ServiceNow platform:

- Users authenticate via standard ServiceNow login (username/password or SSO, depending on instance configuration)
- Service Portal session is maintained via ServiceNow's native session cookies
- Catalog item visibility and submission rights are controlled through the item's Availability and Permission tabs (roles/ACLs)
- No custom token-based authentication was implemented, as the entire solution runs within the authenticated ServiceNow session

9. User Interface

Service Portal Home

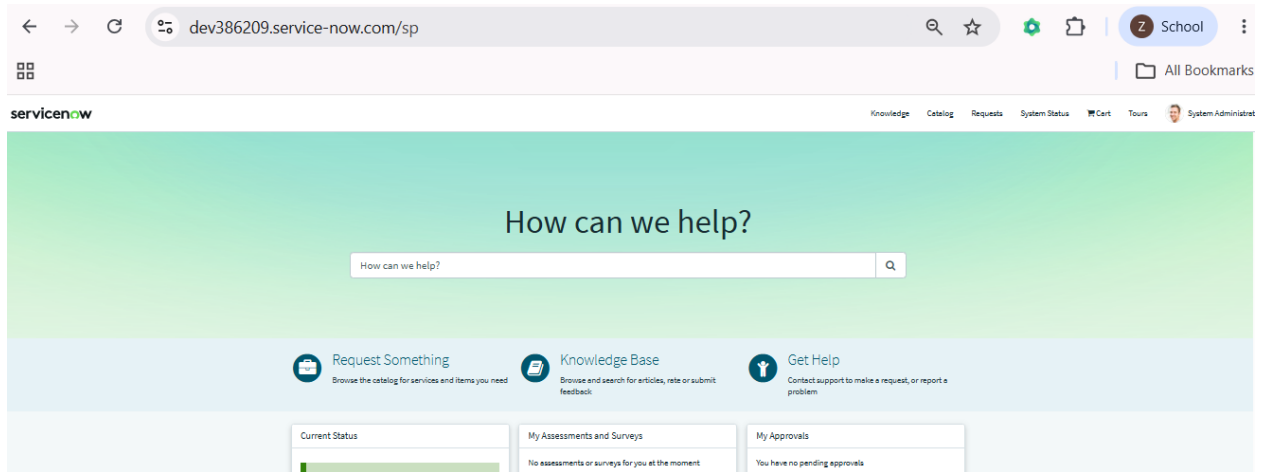


Figure 1: Service Portal home page — 'How can we help?' search

Catalog Item — Initial View

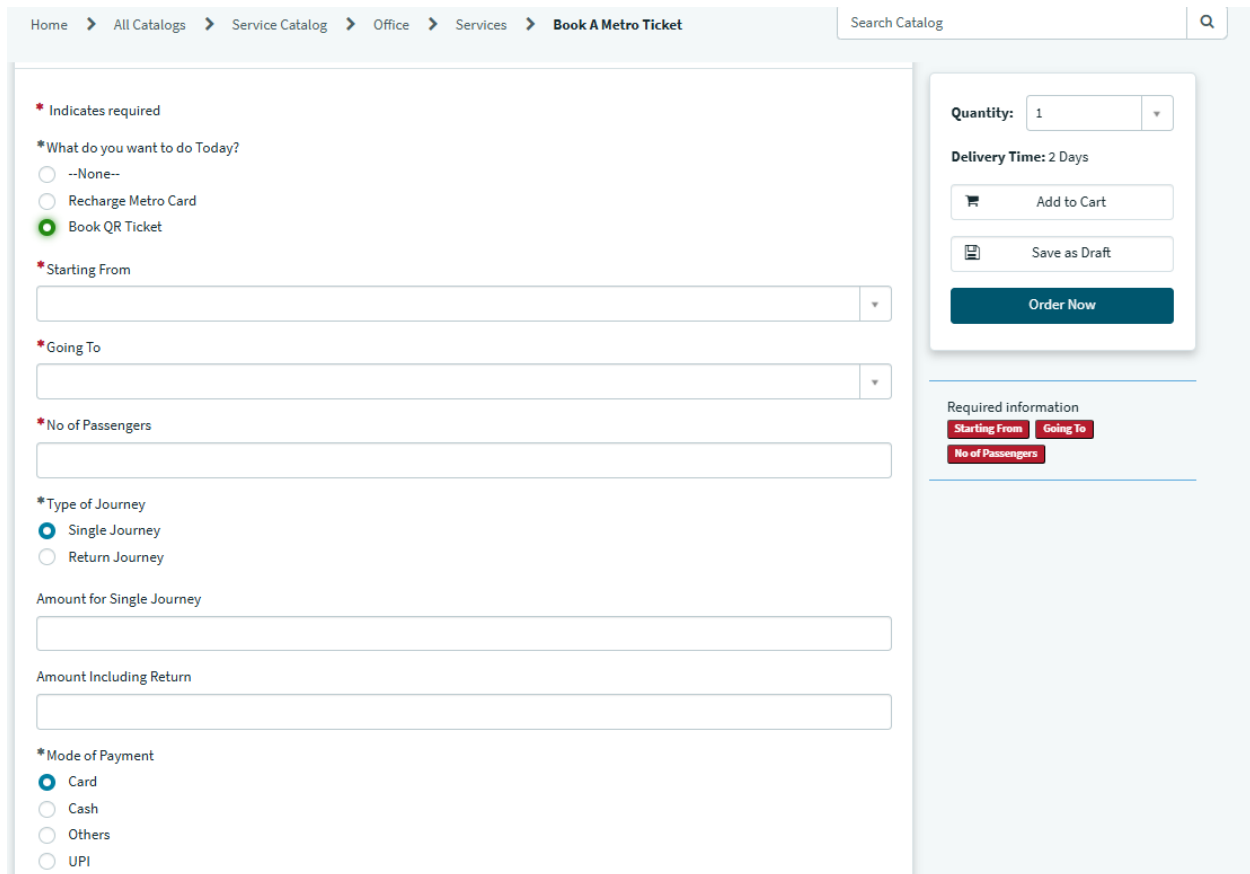


Figure 2: Book A Metro Ticket catalog item with branching choice

Fill the form

Home > All Catalogs > Service Catalog > Office > Services > Book A Metro Ticket

Search Catalog

* Indicates required

*What do you want to do Today?

☐ --None--

☐ Recharge Metro Card

☒ Book QR Ticket

*Starting From

Ameerpet

X

▼

*Going To

LB Nagar

X

▼

*No of Passengers

2

*Type of Journey

☐ Single Journey

☒ Return Journey

Amount for Single Journey

Amount Including Return

200

*Mode of Payment

☐ Card

☒ Cash

☐ Others

☐ UPI

Quantity: 1

▼

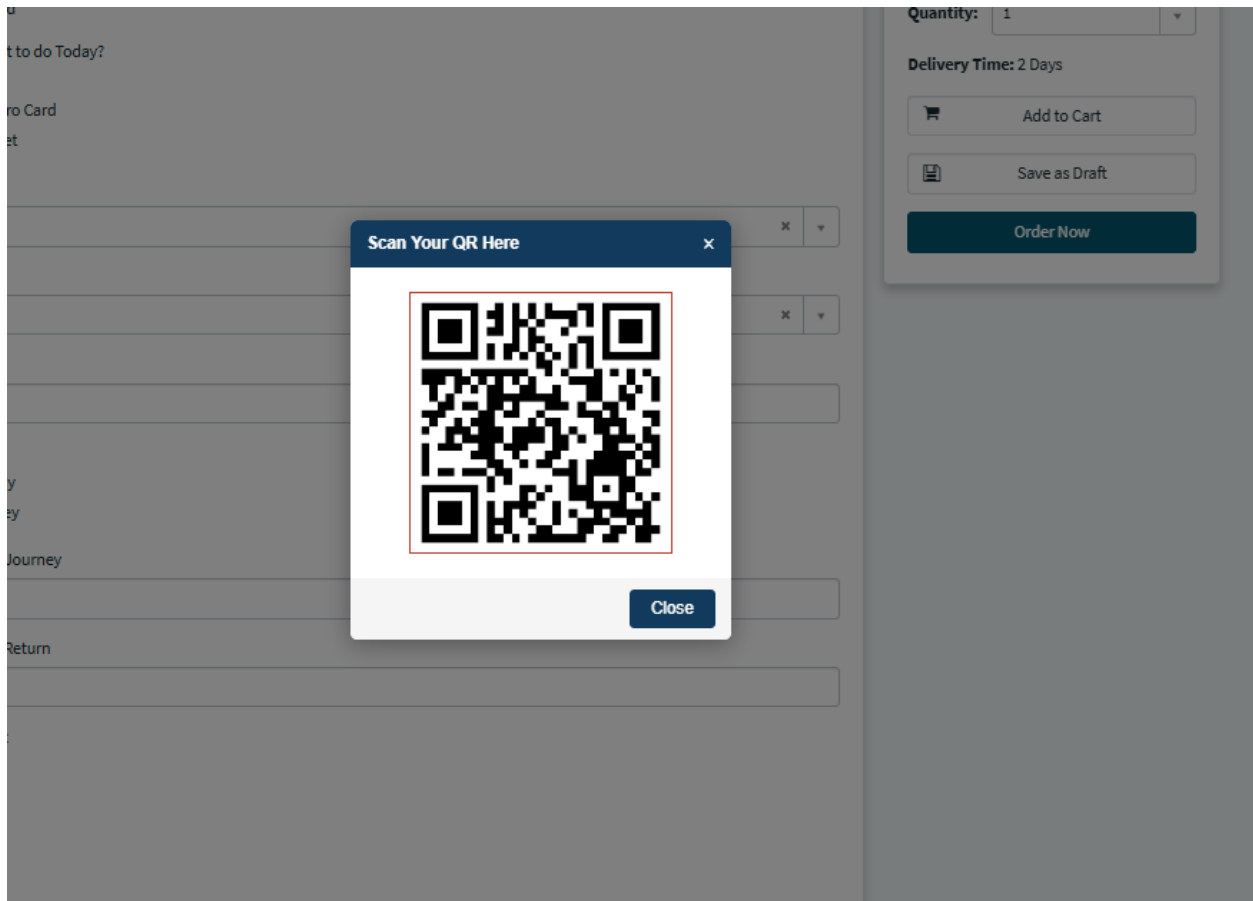
Delivery Time: 2 Days

 Add to Cart

 Save as Draft

Order Now

Click order now



Click close and then click order now to submit the request

Home > Request Summary - REQ0010049

Submitted : 2026-07-31 10:30:25
Request Number : REQ0010049
Estimated Delivery : 2026-08-02

Item	Delivery Date	Stage	Price (each)	Quantity	Total
Book A Metro Ticket	2026-08-02	▶ Assess or Scope Task	---	1	---
					Total: \$0.00

Book a Metro Ticket — Journey Details

Home > My Request - RITM0010051

A metro e-ticketing system allows passengers to purchase and use tickets digitally, typically via a mobile app or website, eliminating the need

RITM0010051 Open Created just now Last updated just now Hide details ^

Item	Requested for	Quantity
Book A Metro Ticket	 System Administrator	1

Stage

▶ Assess or Scope Task

Activity Attachments Additional Details

qr_confirmed
true

What do you want to do Today?
Book QR Ticket

Starting From
Ameerpet

Going To
L B Nagar

No of Passengers
2

Type of Journey
Return Journey

Amount Including Return
200

Mode of Payment
Cash

Figure 3: Journey fields revealed after selecting Book QR Ticket

Fare Auto-Calculation

Instructions required

*What do you want to do Today?

☐ --None--

☐ Recharge Metro Card

☒ Book QR Ticket

*Starting From

*Going To

*No of Passengers

*Type of Journey

☐ Single Journey

☒ Return Journey

Amount for Single Journey

Amount Including Return

*Mode of Payment

☒ Card

☐ Cash

☐ Others

☐ UPI

Add attachments

Figure 4: Auto-calculated fare and payment mode with 'Others' free-text field

Recharge Smart Card — Field View

The screenshot shows the 'Book A Metro Ticket' form in the ServiceNow interface. The breadcrumb trail is: Home > All Catalogs > Service Catalog > Office > Services > Book A Metro Ticket. The form title is 'Book A Metro Ticket' with a description: 'A metro e-ticketing system allows passengers to purchase and use tickets digitally, typically via a mobile app or website, eliminating the need'. The form contains several required fields marked with a red asterisk: 'What do you want to do Today?' with radio button options for '--None--', 'Recharge Metro Card' (selected), and 'Book QR Ticket'; 'Enter Smart Card Number' with a text input field containing 'e.g. 1234567890'; 'Enter Smart Card Name' with a text input field; and 'Recharge Amount' with a text input field. To the right, there is a sidebar with 'Quantity: 1', 'Delivery Time: 2 Days', and buttons for 'Add to Cart', 'Save as Draft', and 'Order Now'. Below the sidebar, a 'Required information' section lists 'Enter Smart Card Number', 'Enter Smart Card Name', and 'Recharge Amount' in red boxes.

Figure 5: Recharge Smart Card fields alongside ticket fields (pre-fix)

Order Confirmation

The screenshot shows the 'Order Confirmation' modal dialog. The title is 'Order Confirmation' with a close button. The form contains: 'Request for' with a dropdown menu showing 'System Administrator'; 'Delivery Information (Optional)' with a plus icon and a text input field; and 'Special instructions (Optional)' with a plus icon and a text input field. At the bottom right, there are 'Cancel' and 'Submitting...' buttons. The background shows a blurred view of the 'Book A Metro Ticket' form.

Figure 6: Order Confirmation modal

Requested Item — Additional Details

Application

Global

Type

Multiple Choice

Catalog Item

Book A Metro Ticket

Order

100

Active

☒

Mandatory

☒

Read only

☐

Hidden

☐

Disable automatic slot fill based on user context

☐

Question

Annotation

Type Specifications

* Default Value

Auto-populate

Permission

Availability

* Default value

Copy

Update

Delete

Related Links

[n Point Scan](#)

[Utils | Versions \(4\)](#)

Question Choices

Order

Search

Actions on selected rows...

New

Question = What do you want to do Today?

Text	Value	Order	Inactive
Recharge Smart Card	recharge_card	200	false
Book a Metro Ticket		100	false

Figure 7: RITM generated with Additional Details tab

Order Status Confirmation

Order Status

Back to Catalog

Continue Shopping

Home

Thank you, your request has been submitted

Order Placed:

2026-07-29 19:37:00

Request Number:

REQ0010021

Estimated Delivery Date of Complete Order:

2026-07-31

Description	Delivery Date	Stage	Price (ea.)	Quantity	Total
A metro e-ticketing system allows passengers to purchase and use tickets digitally, typically via a mobile app or website, eliminating the need	2026-07-31			1	
				Total	-

Back to Catalog

Continue Shopping

Home

Figure 8: Order successfully submitted — REQ0010021

Request Summary

Home > Request Summary - REQ0010022					
Submitted : 2026-07-29 19:59:47 Request Number : REQ0010022 Estimated Delivery : 2026-07-31					
Item	Delivery Date	Stage	Price (each)	Quantity	Total
Book A Metro Ticket	2026-07-31	Request Approval	---	1	---
					Total: \$0.00

Figure 9: Final Request Summary — REQ0010022

When we fill the recharge card form and click order now the records will be automatically created in metro database table

Metro Databases Mode of Payment Search					
All	Mode of Payment	Recharge Amount	Smart Card Name	Smart Card Number	User Details
	Search	Search	Search	Search	Search
	BHMI UPI	500	RUPAY	1234567890	System Administrator
					2026-07-29 23:30:19

10. Testing

Testing Strategy

Manual functional testing was performed directly on the ServiceNow PDI Service Portal, covering both branches of the catalog item.

Test Cases Executed

#	Test Case	Result
1	Book a Metro Ticket — Single Journey fare calculation	Pass
2	Book a Metro Ticket — Return Journey fare calculation	Pass
3	Payment Mode = Others reveals free-text field	Pass
4	Order submission generates RITM with correct Additional Details	Pass
5	Recharge Smart Card — UI Policy hides ticket fields	Fail (fixed — see Known Issues)
6	QR code generation on Recharge Smart Card submission	Fail (fixed — GlideModal replaced with window.open)

Tools Used

- Browser DevTools (Console & Elements tabs) — used to diagnose the GlideModal reference error
- ServiceNow PDI native testing — direct Service Portal order submission

11. Screenshots

All key screens are captured in Section 9 above, covering the full navigation flow from Service Portal home through to Order Confirmation and Request Summary.

12. Known Issues

Resolved During Development

- UI Policy for 'Recharge Smart Card' did not hide the ticket-booking fields — root cause was a mismatch between the UI Policy condition and the actual Question Choice Value; corrected by verifying the exact Value column in Question Choices.
- QR popup showed 'Page not found' — root cause was use of GlideModal referencing a non-existent UI Page (external_qr_api_page); this API is Classic UI-only and is not available in Service Portal.
- QR Generation script threw 'ReferenceError: GlideModal is not defined' in the Service Portal — resolved by replacing GlideModal with a native window.open() popup rendering the QR Server API image directly.

Outstanding

- Browser popup blockers may prevent the QR window from opening on first attempt — requires the user to allow popups for the instance domain.
- QR Server API is an external third-party domain; outbound access must be confirmed on restricted/enterprise instances.

13. Future Enhancements

- Replace the third-party QR Server API with an in-platform QR generation library to remove the external dependency
- Display the QR code inline within the Requested Item's Additional Details view (via a Service Portal widget) instead of a separate popup window
- Add Flow Designer automation to route Recharge Smart Card requests to a fulfillment/approval flow distinct from ticket bookings
- Add mandatory attachment validation (e.g., payment proof) only when Mode of Payment is UPI or Others
- Add real fare-matrix lookup (via a Data Lookup Definition) instead of a flat per-passenger rate